UNIT 4 : Advanced Client-Side Programming – React JS

Topics

1. React JS – Introduction
2. ReactDOM
3. JSX – JavaScript XML
4. Components
5. Properties (Props)
6. State and Lifecycle
7. Events in React
8. Fetch API in React
9. JS Local Storage
10. Lifting State Up
11. Composition and Inheritance

1. React JS – Introduction

1.1 What is React JS?

React JS is an open-source JavaScript **library** developed and maintained by **Facebook (Meta)** and first released in **2013**. It is specifically designed for building fast, interactive, and dynamic **User Interfaces (UI)** for web applications. React is NOT a full framework — it manages only the **View layer** of a web application (what the user sees on screen). For routing, data management, and backend communication, additional libraries or tools are required.

Definition: React JS

React JS is an open-source, component-based JavaScript library for building fast and interactive user interfaces. It employs a Virtual DOM to minimize direct browser DOM manipulation, resulting in highly efficient rendering. React follows a declarative programming model — the developer describes the desired UI state, and React handles all underlying DOM updates automatically.

1.2 Key Features of React JS

- **Component-Based Architecture:** Every part of a React application — a button, a form, a header, a card — is a Component. Components are independent, reusable pieces of UI that can be combined to build complex interfaces.
- **Declarative Style:** In declarative programming, you describe what the UI should look like for a given state, and React takes care of how to update and render the DOM efficiently. This is in contrast to imperative programming where you manually manipulate the DOM.
- **Virtual DOM:** React maintains a lightweight in-memory representation of the real browser DOM, called the Virtual DOM. When application state changes, React first updates the Virtual DOM, computes the difference (diffing), and then updates only the affected parts of the real DOM. This process is called Reconciliation, and it makes React significantly faster than traditional DOM manipulation.
- **Unidirectional Data Flow:** Data in React flows in one direction: from Parent components to Child components via props. This makes the application easier to debug and reason about.
- **React Hooks:** Hooks (introduced in React 16.8) are special functions that let functional components use state and lifecycle features — eliminating the need for class components in most cases.
- **Rich Ecosystem:** React has a vast ecosystem of supporting libraries: React Router (routing), Redux/Context API (state management), Axios (HTTP requests), and React Native (mobile development).

1.3 Why React? – Advantages and Real-World Applications

React has become the most widely used front-end library in the world due to the following advantages:

- Fast rendering performance using the Virtual DOM — only changed parts of the UI are updated.
- Reusable components save significant development time across large projects.
- Strong community support with millions of developers and rich third-party library ecosystem.
- React Native allows developers to use the same React knowledge to build native mobile applications for iOS and Android.
- Easy integration with REST APIs and backend servers for building full-stack applications.
- SEO-friendly when combined with server-side rendering frameworks like Next.js.

Real-World Applications Built with React JS:

Facebook & Instagram: The social media giants — React was originally built by Facebook for its own News Feed UI.

Netflix: Netflix rebuilt their UI with React for improved performance and faster rendering.

Flipkart / Swiggy: Major Indian e-commerce and food delivery platforms use React for fast, interactive UIs.

Notion: The popular productivity and note-taking tool is built entirely on React.

1.4 React JS vs Traditional JavaScript (DOM Manipulation)

Feature	Traditional JavaScript	React JS
UI Updates	Manually update DOM with getElementById, innerHTML, etc.	Automatically re-renders only changed components
Code Organisation	Procedural, can become messy for large apps	Component-based — modular and maintainable
Performance	Entire DOM re-rendered on changes	Virtual DOM ensures only changed nodes update
Reusability	Limited — code is often duplicated	High — components can be reused across the app

2. ReactDOM

2.1 What is ReactDOM?

ReactDOM is a separate package from the core React library. While React deals with creating and managing the component tree (in memory), **ReactDOM** is the bridge that connects the React component tree to the actual browser DOM — making the components visible to the user on the webpage.

Definition: ReactDOM

ReactDOM is a package in the React ecosystem that provides the necessary methods to render React components into the actual browser Document Object Model (DOM). It acts as the bridge between the React Virtual DOM (in memory) and the real browser DOM. The most important method it provides is createRoot() (React 18+) and render(), which mount the root React component into a specified HTML element.

2.2 The Virtual DOM — Concept and Working

The **Virtual DOM** is a lightweight, in-memory JavaScript representation of the actual browser DOM. React uses the Virtual DOM to improve performance by minimizing the number of direct browser DOM operations — which are slow and expensive.

How the Virtual DOM Works (Step-by-Step):

Step 1: The application's state or data changes (e.g., user clicks a button).

Step 2: React updates the Virtual DOM first (not the real DOM).

Step 3: React then performs Diffing — it compares the new Virtual DOM with the previous Virtual DOM snapshot to find exactly what has changed.

Step 4: React determines the minimal set of changes required (called the changeset).

Step 5: React performs Reconciliation — it updates only the changed nodes in the real browser DOM.

Step 6: The browser re-paints only the affected parts of the page.

2.3 ReactDOM Code — index.js (Entry Point)

Every React application has an entry point file called **index.js** (or **main.jsx** in Vite). This file is responsible for mounting the root React component into the HTML page. Below is the standard code used in every React application:

```
// index.js – Entry point of every React application
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';

// createRoot() is the modern React 18 approach
// document.getElementById('root') selects the <div id='root'> in index.html
const root = ReactDOM.createRoot(document.getElementById('root'));

// root.render() mounts the <App /> component into the 'root' div
root.render(<App />);

// The 'root' div is defined in public/index.html
// ReactDOM mounts the entire React component tree inside that <div id='root'>
```

Key Point: The tag in public/index.html is the single HTML element where the entire React application is injected. React takes full control of this element and manages everything inside it dynamically.

3. JSX – JavaScript XML

3.1 What is JSX?

JSX stands for **JavaScript XML**. It is a syntax extension for JavaScript that allows developers to write HTML-like code directly inside JavaScript files. JSX is **NOT** plain HTML — it is a syntactic shorthand that gets transpiled (converted) to regular JavaScript **React.createElement()** calls by a tool called **Babel** before the browser executes it. This makes React code much more readable and intuitive to write.

Definition: JSX (JavaScript XML)

JSX is a syntax extension for JavaScript, used primarily with React, that allows developers to write HTML-like markup directly within JavaScript code. JSX is transpiled to standard JavaScript (specifically React.createElement() function calls) by build tools such as Babel before execution in the browser. JSX is not mandatory in React, but it is strongly recommended due to the significant improvement in code readability and developer productivity it provides.

3.2 JSX vs Plain JavaScript — Comparison

To understand the importance of JSX, compare the same UI written with and without JSX:

Without JSX (harder to read):

```
// Without JSX – must use React.createElement() for every element
const element = React.createElement(
  'h1',
  { className: 'title' },
  'Hello, Student!'
);
```

With JSX (clear and readable — React way):

```
// With JSX – looks like HTML, much more readable!
const element = (
  <h1 className='title'>
    Hello, Student!
  </h1>
);

// Babel converts the above JSX into the React.createElement() call
automatically.
```

3.3 Important Rules of JSX

JSX is similar to HTML but has several important rules that must be followed:

Rule 1: Return One Root Element

A JSX expression must always return a single root element. You cannot return two sibling elements at the same level. Wrap them in a or use a React Fragment (<> </>) to group them without adding extra HTML.

Rule 2: Use className Instead of class

Since JSX is JavaScript, the word class is a reserved keyword. Use className instead when applying CSS classes.

Rule 3: Self-Close Empty Tags

Every tag in JSX must be properly closed. Tags without content must be self-closed with a forward slash before the closing angle bracket:
, <img />, <input />

Rule 4: JavaScript Expressions Inside { }

To embed JavaScript variables, expressions, or function calls inside JSX, place them inside curly braces { }. You can use ternary operators, array methods, and arithmetic expressions inside { }.

```
// Rule 1: Return one root element
return (
  <div>
    <h1>Title</h1>
    <p>Text</p>
  </div>
);

// Rule 2: Use className (not class)
<div className='box'>Hello</div>

// Rule 3: Self-close empty tags
<img src='photo.jpg' />
<br />
<input type='text' />

// Rule 4: JS expressions in { }
const name = 'Raj';
<h2>Hello, {name}!</h2>
<p>Sum: {2 + 3}</p>
```

3.4 Embedding Expressions and Rendering Lists in JSX

```
// JSX – Embedding expressions and ternary conditions
const student = 'Akash';
const score = 88;
return (
  <div>
    <h2>Welcome, {student}!</h2>
    <p>Your score: {score} / 100</p>
    <p>Grade: {score >= 90 ? 'A' : score >= 75 ? 'B' : 'C'}</p>
  </div>
);

// Rendering a List with map()
const subjects = ['Math', 'React', 'Node'];
return (
  <ul>
    {subjects.map((sub, index) => (
      <li key={index}>{sub}</li>
    ))}
  </ul>
);
// 'key' prop is REQUIRED when rendering lists using map()
```

Important: Always provide a unique key prop when rendering lists. The key helps React efficiently identify which list items have changed, been added, or been removed. Using the array index as key is acceptable for static lists, but for dynamic lists, prefer using a unique identifier like an ID from the database.

4. Components

4.1 What is a Component?

Definition: Component

In React, a Component is an independent, reusable, and self-contained piece of User Interface (UI). Every React application is built by assembling multiple components together, similar to assembling LEGO blocks. A component encapsulates its own structure (JSX/HTML), style (CSS), and behavior (JavaScript logic). Components can be nested inside other components, creating a parent-child hierarchy that forms the overall application structure.

Think of a webpage as a collection of blocks. A typical React application might have components such as **Header**, **Navbar**, **MainContent**, **ProductCard**, **Footer**, and so on. Each is independent and reusable.

4.2 Functional Components (Modern Approach)

Definition: Functional Component

A Functional Component is a plain JavaScript function that accepts an optional props parameter and returns JSX to describe the UI. Functional components are the modern, recommended way to write React components. Since the introduction of React Hooks (React 16.8), functional components can manage state, handle side effects, and access lifecycle features — making class components largely unnecessary for new development.

```
// Basic Functional Component
import React from 'react';

function Greeting() {
  return (
    <div>
      <h1>Hello, Welcome to React!</h1>
      <p>This is a functional component.</p>
    </div>
  );
}
export default Greeting;

// Arrow Function Style (also valid)
const StudentCard = () => {
  return (
    <div className='card'>
      <h3>Student Name: Raj</h3>
```

```
      <p>Semester: 2  |  Branch: MCA</p>
    </div>
  );
};
export default StudentCard;
```

4.3 Class Components

A **Class Component** is defined using an ES6 class that extends **React.Component**. Before React Hooks were introduced in React 16.8, class components were the only way to manage state and lifecycle. While functional components are now strongly preferred for new projects, understanding class components is essential because many existing and legacy codebases use them.

```
// Welcome.js – Class Component
import React, { Component } from 'react';

class Welcome extends Component {
  // Constructor: called when the component is first created
  constructor(props) {
    super(props);           // Always call super(props) first
    this.state = {          // Initialize state here
      message: 'Hello from Class Component!',
    };
  }
  // render() method MUST return JSX
  render() {
    return (
      <div>
        <h1>{this.state.message}</h1>
        <p>Props received: {this.props.name}</p>
      </div>
    );
  }
}
export default Welcome;

// App.js – Using the Class Component
import React from 'react';
import Welcome from './Welcome';

function App() {
  return (
    <div> <Welcome name='Jay' /> </div>
  );
}
export default App;
```

4.4 Functional vs Class Components — Comparison

Feature	Functional Component	Class Component
Syntax	Plain JavaScript function	ES6 Class extending Component
State Management	useState Hook	this.state and this.setState()
Lifecycle	useEffect Hook	componentDidMount, componentDidUpdate, componentWillUnmount
Code Length	Shorter and cleaner	More verbose (boilerplate code)
Recommended	Yes — modern approach	Legacy — still valid

4.5 Component Reusability and Nesting

One of the greatest strengths of React is **component reusability**. Once a component is created, it can be used multiple times throughout the application, just like calling a function. Components are nested by including them as JSX tags inside other components, creating parent-child relationships.

```
// ProductCard.js – A reusable component
function ProductCard({ name, price }) {
  return (
    <div className='card'>
      <h3>{name}</h3>
      <p>Price: Rs. {price}</p>
    </div>
  );
}

// App.js – Using ProductCard multiple times
function App() {
  return (
    <div>
      <ProductCard name='Laptop' price={45000} />
      <ProductCard name='Headphones' price={2500} />
    </div>
  );
}
```

5. Properties (Props)

5.1 What are Props?

Definition: Props (Properties)

Props, short for Properties, are the mechanism by which data is passed from a Parent component to a Child component in React. Props work exactly like function arguments — the parent passes values, and the child receives and uses them. A fundamental rule in React is that Props are read-only (immutable). A child component must never modify its own props directly. If data needs to flow upward, callback functions are passed as props.

Props enable **unidirectional data flow** in React — data always moves from parent to child. This makes the application predictable and easier to debug.

5.2 Sending and Receiving Props

```
// App.js – Parent component sends props
function App() {
  return (
    <div>
      <StudentInfo
        name='Jeet Mehta'   // string prop
        rollNo={101}         // number prop (use { } for non-string values)
        branch='MCA'
      />
    </div>
  );
}

// StudentInfo.js – Child component receives props
function StudentInfo(props) {
  return (
    <div className='student-card'>
      <h3>Name: {props.name}</h3>
      <p>Roll No: {props.rollNo}</p>
      <p>Branch: {props.branch}</p>
    </div>
  );
}
```

Important: In JSX, string values can be passed directly using double quotes: name="Raj". All other data types — numbers, booleans, arrays, objects, and functions — must be passed inside curly braces { }.

5.3 Props Destructuring and Default Values

Instead of repeatedly writing **props.name**, **props.price**, etc., it is considered best practice to **destructure** props directly in the function parameter. Default values can also be set using the **=** operator inside the destructuring syntax.

```
// Destructuring props in function parameter
function ProductCard({ name, price, available = true }) {
  // 'available = true' sets a DEFAULT PROP value
  // If the parent does not pass 'available', it defaults to true
  return (
    <div>
      <h3>{name}</h3>
      <p>Price: Rs. {price}</p>
      <p>Status: {available ? 'In Stock' : 'Out of Stock'}</p>
    </div>
  );
}

// Usage in parent:
<ProductCard name='Phone' price={15000} />
// 'available' is not passed, so it defaults to true
```

5.4 Types of Data That Can Be Passed as Props

Data Type	Syntax Example	Description
String	name="Raj"	Pass directly using quotes
Number	age={20}	Must use curly braces
Boolean	isActive={true}	true or false in curly braces
Array	items={[1,2,3]}	Array literal in curly braces
Object	user={{ id:1, name:'Raj' }}	Double curly braces (outer = JSX, inner = object)
Function	onClick={handleClick}	Pass function reference (used for callbacks)

6. State and Lifecycle

6.1 What is State?

Definition: State

State is a built-in React object that stores data or information specific to a component. When a component's state changes, React automatically re-renders the component to reflect the updated data in the UI. State is different from Props in two important ways: (1) State is defined and managed inside the component itself, while Props are passed from outside (parent). (2) State is mutable — it can change over time in response to user actions or events. Props are immutable — they cannot be modified by the receiving component.

6.2 useState Hook — Managing State in Functional Components

The **useState** Hook is the standard way to add state to a functional component. It returns an array of exactly two values: the **current state value** and a **setter function** to update it. When the setter function is called, React re-renders the component with the new state value.

```
import React, { useState } from 'react';

function Counter() {
  // useState(0) : initializes count with value 0
  // count       : current state value (read only — never modify directly)
  // setCount    : function to UPDATE the state
  const [count, setCount] = useState(0);

  return (
    <div>
      <h2>You clicked {count} times</h2>
      <button onClick={() => setCount(count + 1)}>
        Click Me +1
      </button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
export default Counter;
```

How useState Works — Step by Step:

Step 1: useState(0) is called — React initializes the count state variable with value 0.

Step 2: React stores the value count = 0 internally for this component.

Step 3: The user clicks the '+1' button.

Step 4: setCount(count + 1) is called — React schedules a state update.

Step 5: React updates the state value to 1.

Step 6: React re-renders the Counter component.

Step 7: The UI now displays: 'You clicked 1 times'.

6.3 Multiple State Variables and Forms

A single component can have **multiple state variables**. Each call to **useState()** creates one independent piece of state. This is the recommended pattern for managing form inputs in React:

```
import React, { useState } from 'react';

function LoginForm() {
  // Multiple state variables — each is independent
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const [isLoggedIn, setIsLoggedIn] = useState(false);

  const handleLogin = () => {
    if (username === 'admin' && password === '1234') {
      setIsLoggedIn(true);
    }
  };

  if (isLoggedIn) {
    return <h2>Welcome, {username}! Login successful.</h2>;
  }

  return (
    <div>
      <input value={username}
             onChange={e => setUsername(e.target.value)}
             placeholder='Enter username' />
      <input value={password}
             onChange={e => setPassword(e.target.value)}
             type='password' placeholder='Enter password' />
      <button onClick={handleLogin}>Login</button>
    </div>
  );
}
```

6.4 Component Lifecycle

Definition: Component Lifecycle

Every React component goes through a series of phases from the time it is created and added to the DOM to the time it is removed. These phases are collectively called the Component Lifecycle. The three main phases are: (1) Mounting — when the component is first created and inserted into the DOM; (2) Updating — when the component's state or props change, causing it to re-render; and (3) Unmounting — when the component is removed from the DOM and destroyed.

Phase	When It Occurs	Class Component Method
Mounting	Component created and added to DOM	constructor() → render() → componentDidMount()
Updating	State or Props change triggers re-render	render() → componentDidUpdate()
Unmounting	Component removed from DOM	componentWillUnmount()

6.5 useEffect Hook — Handling Lifecycle in Functional Components

Definition: useEffect Hook

useEffect is a React Hook that allows functional components to perform side effects — operations that occur outside the normal render cycle, such as fetching data from an API, setting up timers, manually updating the DOM, or subscribing to events. useEffect accepts two arguments: (1) a callback function containing the side-effect logic, and (2) an optional dependency array that controls when the effect runs. The callback can optionally return a cleanup function that runs when the component unmounts.

```
import React, { useState, useEffect } from 'react';

function Timer() {
  const [seconds, setSeconds] = useState(0);

  // useEffect with empty [] runs ONCE after the first render (Mounting)
  // Equivalent to componentDidMount in class components
  useEffect(() => {
    const interval = setInterval(() => {
      setSeconds(prev => prev + 1);
    }, 1000);

    // Cleanup function: runs when component Unmounts
    // Equivalent to componentWillUnmount in class components
    return () => clearInterval(interval);
  }, []);  // Empty [] = run only ONCE on mount
```

```
  return <h2>Timer: {seconds} seconds</h2>;
}
```

useEffect Dependency Array — Controlling When the Effect Runs:

Dependency Array	When useEffect Runs	Equivalent To (Class)
Not provided	After EVERY render (mount + every update)	componentDidMount + componentDidUpdate
Empty [ ]	Only ONCE after the first render (mount)	componentDidMount only
[variable]	After mount + whenever 'variable' changes	componentDidUpdate (conditional)

7. Events in React

7.1 What are Events in React?

In React, **events** are handled very similarly to HTML DOM events, but with important syntactic differences. React uses **Synthetic Events** — a wrapper around the browser's native events that normalizes event behaviour across different browsers, ensuring consistent behavior in all environments.

7.2 HTML Events vs React Events — Key Differences

Feature	HTML (Traditional)	React
Event Naming	lowercase: onclick, onchange	camelCase: onClick, onChange
Handler Value	String: onclick="handleClick()"	Function reference: onClick={handleClick}
Prevent Default	Use return false	Call e.preventDefault()
Event Object	window.event (global)	Passed as parameter: (e) or (event)

7.3 Common React Events with Code Example

```
function EventDemo() {
  // onClick – fires when button is clicked
  const handleClick = () => alert('Button clicked!');

  // onChange – fires when input value changes
  const handleChange = (e) => console.log('Input:', e.target.value);

  // onSubmit – fires when form is submitted
  const handleSubmit = (e) => {
    e.preventDefault();  // Prevents the page from reloading
    alert('Form submitted!');
  };

  return (
    <form onSubmit={handleSubmit}>
      <input type='text' onChange={handleChange} placeholder='Type here' />
      <button type='submit' onClick={handleClick}>Submit</button>
    </form>
  );
}
```

7.4 Commonly Used React Event Handlers

Event Handler	Triggered When	Common Use
onClick	User clicks an element	Buttons, links, cards
onChange	Input value changes	Text inputs, dropdowns, checkboxes
onSubmit	Form is submitted	Login forms, registration forms
onMouseOver	Mouse hovers over an element	Tooltips, hover effects
onKeyDown	Key is pressed	Keyboard shortcuts, search as you type
onFocus / onBlur	Input gains / loses focus	Form validation, highlight active fields

8. Fetch API in React

8.1 What is the Fetch API?

Definition: Fetch API

The Fetch API is a modern, Promise-based browser interface that provides a way to make HTTP requests from client-side JavaScript. It is built into all modern browsers and does not require any additional library or installation. In React, the Fetch API is commonly used inside the useEffect Hook to retrieve data from external APIs or backend servers when a component mounts. It returns a Promise that resolves to a Response object, from which the data can be extracted in JSON or other formats.

8.2 Fetching Data Using .then() / .catch()

The following example demonstrates how to fetch a list of users from a public API and display them using React state:

```
import React, { useState, useEffect } from 'react';

function UserList() {
  const [users, setUsers] = useState([]);       // Store list of users
  const [loading, setLoading] = useState(true);  // Loading indicator

  useEffect(() => {
    // Fetch data from a public test API
    fetch('https://jsonplaceholder.typicode.com/users')
      .then(response => response.json())   // Parse the JSON response body
      .then(data => {
        setUsers(data);           // Store fetched data in state
        setLoading(false);        // Hide loading indicator
      })
      .catch(error => console.error('Error fetching data:', error));
  }, []);  // Empty [] = run only ONCE when component mounts

  if (loading) return <p>Loading...</p>;

  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>{user.name} - {user.email}</li>
      ))}
    </ul>
  );
}
```

8.3 Fetching Data Using async/await (Recommended)

Using **async/await** inside **useEffect** makes the code cleaner and easier to read. Note that **useEffect's callback cannot be async directly** — instead, an inner async function is defined and immediately called:

```
import React, { useState, useEffect } from 'react';

function PostList() {
  const [posts, setPosts] = useState([]);
  const [error, setError] = useState(null);

  useEffect(() => {
    // Define an async function INSIDE useEffect
    const fetchPosts = async () => {
      try {
        const response = await fetch(
          'https://jsonplaceholder.typicode.com/posts?_limit=5'
        );
        const data = await response.json();
        setPosts(data);
      } catch (err) {
        setError('Failed to load posts. Please try again.');
      }
    };
    fetchPosts();  // Immediately call the async function
  }, []);

  if (error) return <p style={{ color: 'red' }}>{error}</p>;
  return <ul>{posts.map(p => <li key={p.id}>{p.title}</li>)}</ul>;
}
```

Why can't useEffect be async directly? The useEffect Hook expects its callback to return either nothing or a cleanup function. An async function always returns a Promise, which would be returned to useEffect and would cause unexpected behaviour. Therefore, always define an inner async function and call it immediately.

8.4 Fetch API with POST Request (Sending Data to Server)

```
const sendData = async () => {
  const newUser = { name: 'Jay', email: 'jay@example.com' };

  const response = await fetch('https://jsonplaceholder.typicode.com/users', {
    method: 'POST',                          // HTTP method
    headers: {
      'Content-Type': 'application/json',    // Tell server we're sending JSON
    },
    body: JSON.stringify(newUser),           // Convert JS object to JSON
string
  });

  const data = await response.json();        // Parse the server's response
  console.log('Created:', data);
};
```

9. JS Local Storage in React

9.1 What is Local Storage?

Definition: Local Storage

Local Storage is a Web Storage API built into modern browsers that allows web applications to store key-value pairs persistently in the user's browser. Unlike React state (which is lost on page refresh) or session storage (which is cleared when the browser tab is closed), data stored in Local Storage persists indefinitely until it is explicitly removed by the application or the user. Local Storage can store approximately 5–10 MB of data depending on the browser. All values stored in Local Storage are strings.

9.2 Local Storage API Methods

```
// Store a string value
localStorage.setItem('username', 'Jeet');

// Retrieve a stored value
const name = localStorage.getItem('username');  // Returns 'Jeet'

// Remove a specific item
localStorage.removeItem('username');

// Clear ALL data stored in Local Storage
localStorage.clear();

// Store an object or array — MUST convert to JSON string
const cartItems = [{ id:1, name:'Laptop', qty:1 }];
localStorage.setItem('cart', JSON.stringify(cartItems));

// Retrieve and parse back to JavaScript object
const storedCart = JSON.parse(localStorage.getItem('cart'));
```

Important: Local Storage can only store strings. To store JavaScript objects or arrays, use JSON.stringify() when saving and JSON.parse() when reading. Forgetting this is a very common mistake.

9.3 Using Local Storage in React — Practical Example

The following example demonstrates how to persist a user's dark mode preference across page refreshes using Local Storage combined with React state:

```
import React, { useState } from 'react';

function DarkModeToggle() {
  // Load saved preference from localStorage when component first renders
  const savedMode = localStorage.getItem('darkMode');
  const [dark, setDark] = useState(savedMode === 'true');

  const toggle = () => {
    const newVal = !dark;
    setDark(newVal);
    // Save updated preference to localStorage
    localStorage.setItem('darkMode', newVal);
  };

  return (
    <div style={{ background: dark ? '#1a1a1a' : '#ffffff',
                  color: dark ? '#ffffff' : '#1a1a1a',
                  padding: '20px' }}>
      <h2>Dark Mode: {dark ? 'ON' : 'OFF'}</h2>
      <button onClick={toggle}>
        {dark ? 'Switch to Light Mode' : 'Switch to Dark Mode'}
      </button>
    </div>
  );
}
// The preference persists even after the user refreshes the page.
```

9.4 Local Storage vs Other Storage Options

Feature	Local Storage	Session Storage	React State
Persistence	Permanent (until cleared)	Until tab is closed	Lost on page refresh
Scope	All tabs of same origin	Single tab only	Single component
Data Type	Strings only	Strings only	Any JS type
Capacity	~5-10 MB	~5 MB	No fixed limit

10. Lifting State Up

10.1 What is Lifting State Up?

Definition: Lifting State Up

Lifting State Up is a React pattern used when two or more sibling components need to share the same piece of data. Since React follows unidirectional data flow (parent to child via props), sibling components cannot directly communicate with each other. The solution is to move (lift) the shared state up to the nearest common parent component. The parent then owns the state and passes it down to each sibling as props. Children communicate changes back to the parent using callback functions passed as props.

10.2 The Problem — Why Lifting State is Needed

Consider a scenario where **Component A** (an input box) and **Component B** (a display box) are siblings. When the user types in Component A, Component B must immediately display what was typed. Since they are siblings, they cannot directly share state. The state must be **lifted to their common parent** — the **App** component.

```
// Parent (App) holds the shared state and passes to both children
import React, { useState } from 'react';

function App() {
  const [text, setText] = useState('');  // Shared state lives here

  return (
    <div>
      {/* Pass setText as a callback to InputBox */}
      <InputBox onChange={setText} />
      {/* Pass text as a prop to DisplayBox */}
      <DisplayBox value={text} />
    </div>
  );
}

// Child A: receives onChange callback, calls it when input changes
function InputBox({ onChange }) {
  return (
    <input
      onChange={e => onChange(e.target.value)}
      placeholder='Type something...'
    />
  );
```

```
}

// Child B: receives the current value as a prop and displays it
function DisplayBox({ value }) {
  return <p>You typed: {value}</p>;
}
```

10.3 Data Flow in Lifting State Up

State flows downward (parent → child) via props, and events flow upward (child → parent) via callback functions.

Direction	Mechanism	Example
Parent → Child	Props	<DisplayBox value={text} />
Child → Parent	Callback function passed as prop	<InputBox onChange={setText} />

Key Point: Never try to pass data directly between sibling components. Always lift the shared state to their common ancestor. This maintains React's unidirectional data flow and keeps the application predictable.

11. Composition and Inheritance

11.1 Composition in React

Definition: Composition

Composition is the design pattern of building complex UIs by combining smaller, simpler components together — similar to assembling LEGO pieces. In React, composition is achieved by nesting components inside each other and using the special children prop to pass UI content into a component from the outside. React's official documentation explicitly recommends Composition over Inheritance for code reuse between components.

11.2 The children Prop — Building Generic Container Components

The **children** prop is a special built-in prop in React that holds the JSX content placed between a component's opening and closing tags. This allows you to build generic, reusable **container components** (like cards, modals, panels) that do not know their content in advance:

```
// Card.js - A generic reusable container component
function Card({ children, title }) {
  return (
    <div className='card'>
      <h2>{title}</h2>
      <div className='card-body'>
        {children}  {/* Renders whatever content is passed between tags */}
      </div>
    </div>
  );
}

// App.js - Composing the Card with different content each time
function App() {
  return (
    <div>
      <Card title='Student Info'>
        <p>Name: Amit</p>
        <p>Course: MCA</p>
        <button>View Profile</button>
      </Card>

      <Card title='Course List'>
        <ul>
          <li>Full Stack Development</li>
          <li>Database Management</li>
```

```
          </ul>
        </Card>
      </div>
    );
}
```

11.3 Inheritance in React — Why It is NOT Recommended

Definition: Inheritance (OOP)

Inheritance is an Object-Oriented Programming concept where a child class acquires the properties and methods of a parent class using the extends keyword. While this is common in languages like Java and C++, the React team explicitly recommends against using Inheritance for component reuse. React components should use Composition instead of creating class hierarchies.

11.4 Composition vs Inheritance — Comparison

Aspect	Composition (Recommended)	Inheritance (Not Recommended in React)
Coupling	Loose coupling — components are independent	Tight coupling — child depends heavily on parent
Flexibility	Very flexible — combine any components freely	Rigid — changing base class breaks derived classes
Code Reuse	Via children prop and component nesting	Via extends keyword
React Recommendation	Strongly recommended by React team	Explicitly discouraged
Maintainability	High — easy to modify individual components	Low — deep hierarchies become confusing

Important Questions

Q1. What is React JS? Explain its key features and advantages over traditional JavaScript DOM manipulation.

Q2. Explain the concept of Virtual DOM in React. How does diffing and reconciliation improve performance?

Q3. What is ReactDOM? Write the standard code for index.js and explain how it mounts a React application into the browser.

Q4. What is JSX? List and explain all important rules of JSX with examples.

Q5. What is the difference between Functional Components and Class Components in React? When would you use each?

Q6. Explain Props in React. How do you pass and receive props? What does 'Props are read-only' mean?

Q7. What is State in React? Explain the useState Hook with a practical example (Counter application).

Q8. What are the three phases of a React Component Lifecycle? Explain the useEffect Hook and how it handles lifecycle in functional components.

Q9. How do you handle events in React? List the differences between HTML event handling and React event handling.

Q10. Write a React component that fetches data from an API using Fetch API with async/await inside useEffect.

Q11. What is Local Storage? How do you store and retrieve objects in Local Storage from a React component?

Q12. Explain Lifting State Up in React with a diagram and code example.

Q13. What is Composition in React? How does the children prop enable composition? Why is Inheritance not recommended in React?

Q14. Differentiate between State and Props in React. Can a child component modify its props? Explain.